Homework 2
(Preparing the way to understand RSA Public Key Encryption)

Let *n* be a positive integer. We want to test *n* for primality, i.e., determine if *n* is prime. Let *a* and *b* be be two positive integers. We observe that if *n* is a composite number (i.e. not a prime number), it cannot be true that *a* * *b* = *n* and both $a > \sqrt{n}$, $b > \sqrt{n}$. Why? Because if both are bigger than the square root of *n*, their product is bigger than *n.* Therefore, a simple algorithm for testing whether a number is prime is to take its square root, round down to the largest integer less than or equal to the square root (the "floor"), and then check every integer from the floor down to 2 inclusive to see if one of them divides *n*. This algorithm can be sped up substantially by a.) first checking if the number is even (divisible by 2); b.) then checking if it is divisible by 3; c.) if neither of those is true, then take the floor of the square root and start testing each integer in descending order down to 5.

1.) Write the numbers 2, 3, 4, …, 25 across the top of a piece of paper; you might need to write small! Circle every even number. Write a box around every number that is divisible by 3. Now, with respect to this list, explain in words why every prime number must equal 6*k*+1 or 6*k*-1 for some integer k.

Now, consider this:

> what is the fastest way to check if a number can be written as either 6k+1 or 6k-1 for some k? Is there a faster way than just verifying it cannot be divided by either 2 or 3?

Yes, there's a faster way, and it's essentially free: **n mod 6 ∈ {1, 5}** is the whole test, and you can get it from a single division.

Any integer is 6q + r with r in {0,1,2,3,4,5}. The remainders 0, 2, 4 give even numbers, and 0 and 3 give multiples of 3, so only r = 1 or r = 5 survive. Since 6k - 1 = 6(k-1) + 5, "6k-1" is just remainder 5. So "is of the form 6k±1" is *equivalent* to "n mod 6 is 1 or 5," and "not divisible by 2 or 3" is the same condition. The two tests are logically identical, so the question is only which is cheapest to compute.

Faster options

1. **One mod, one comparison.** Compute `r = n % 6` and check `r == 1 || r == 5`. This replaces two divisibility tests with one operation.

2.) Use Claude to write a simple python program to test whether a number is Prime. Force claude to use what you learned in the paragraph at the start of this problem set AND from problem 1. Verify claude implements the algortihm in this way! I will post some numbers on Piazza to test for primeness

3.) Use Claude to write a program to estimate the average number of times that a random number must be generated within a specific interval before you find one that is prime. Do this as follows: generate a random number in the interval 5 million to 6 million, test whether it is prime using your algorithm above (force Claude to re-use that code). Count how many numbers must be generated before you find one that is prime. Repeat this process 1000 times,

and then average the results together. Next, generate a random number between 500 billion and 500 billion plus 1 million and average over 250 attempts for this range. What is your average?

4.) Now, imagine starting with $k = 1$, then letting k increment by 1, and testing sequentially <u>all</u> numbers that satisfy $6k+1$ and $6k-1$. You will test <u>all</u> numbers that might conceievably be prime if you do this. Use Claude to implement this $6k+1$ and $6k-1$ approach in python for finding <u>all</u> the prime numbers, and to create a graph that prints the percentage of numbers that are prime on the interval [0, N] as N varies over the range 5,000 to 5,000,000. Obviously, submit your graph. You may need to force Claude to be efficient (don't let it restart from 0 for every N. Think about this). What do you observe from your graph?

The Euler Totient function of n, denoted by $\Phi(n)$, gives the number of integers less than n whose greatest common divisor with n is 1. Example: $\Phi(12) = 4$. The Euclidean algorithm is an efficient way to determine the greatest common divisor of two numbers, which we denote by gcd(a, b).

Assume $a > b$. The Euclidean algorithm works like this:

- *a.)* Divide a by b and express the result as $a = q_1 b + r_1$ (in python arithmetic: $q_1 = a//b$ and $r_1 = a\%b$). Note that the gcd(a, b), whatever it is, must divide both b and r_1 . Furthermore, the gcd(b, r_1), whatever it is, must also divide a. Therefore it must be true that gcd(a, b) = gcd(b, r_1)
- *b.)* Now, divide b by r_1 to get $b = q_2 r_1 + r_2$. By the exact same argument as above, the gcd(b, r_1) = gcd(r_1, r_2) .
- *c.)* Continue: express $r_1 = q_3 r_2 + r_3$. Now gcd(r_1, r_2) = gcd(r_2, r_3)
- *d.)* Termination: the iteration ends when $r_{n-1} = q_{n+1} r_n + 0$ at which point gcd(r_{n-1}, r_n) = r_n and by following the chain backwards: gcd(a, b) = r_n
- *e.)* Here's a recursive python program for computing the gcd(a, b) (you can use this for testing purposes. Unfortunately, recursion can require a large stack memory when many recursive calls are needed)

```
def gcd(a, b):
  if a%b == 0 : return( b )
  else: return( gcd(b, a%b) )
```

5.) Use Claude to write a <u>non-recursive</u> python function for computing the gcd(a, b)
6.) Use Claude to write a python function for computing the EulerTotient(n) that uses the non-recursive gcd( ) function

7.) Find 2 prime numbers between 2000 and 3000. Call them p and q. Compute, using your function, EulerTotient(pq). Is the result (p-1)(q-1)? Test it for a few combinations